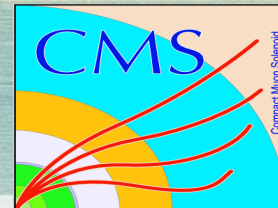


22nd International Conference on Computing in High Energy and Nuclear Physics, Hosted by SLAC and LBNL, Fall 2016



Stability and scalability of the CMS Global Pool: Pushing HTCondor and glideinWMS to new limits

Antonio Pérez-Calero Yzquierdo

on behalf of the CMS Collaboration, Computing and Offline, Submission Infrastructure Group

**CHEP 2016 San Francisco, USA
11th October, 2016**

Ciemat
Centro de Investigaciones
Energéticas, Medioambientales
y Tecnológicas





Outline



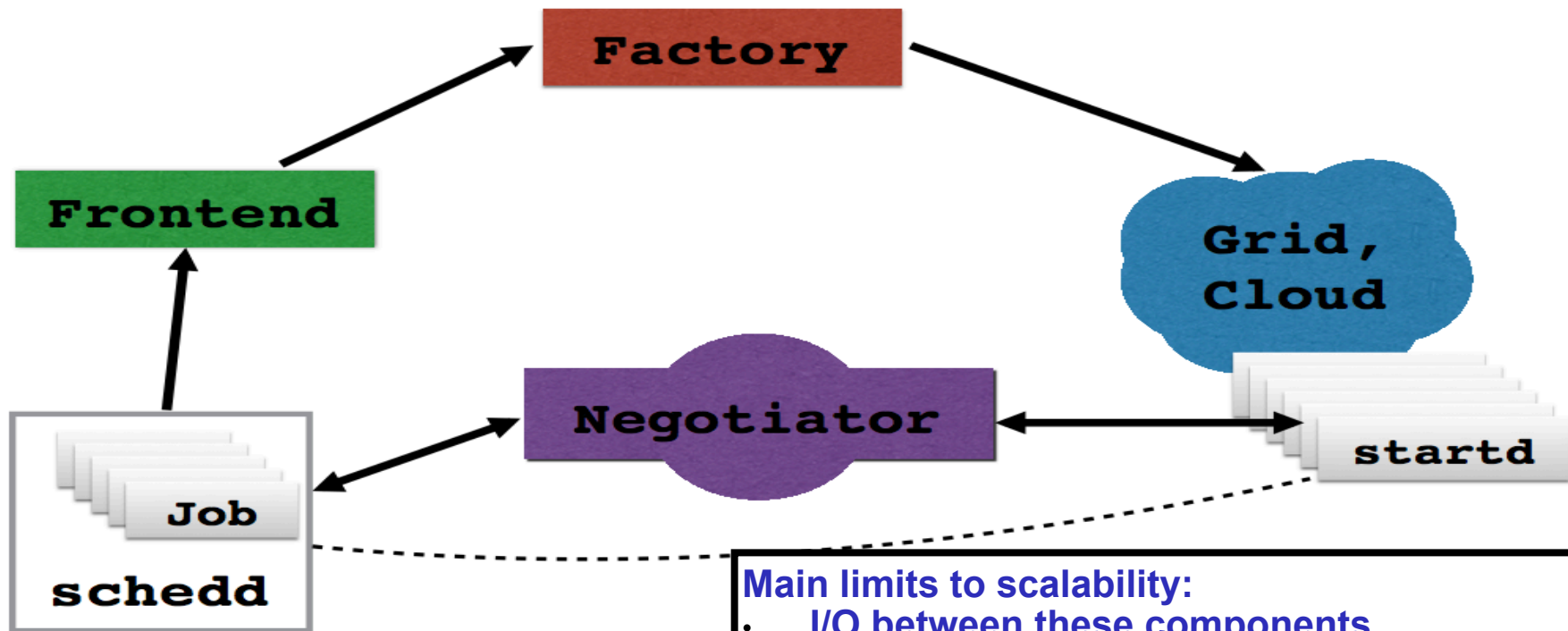
- **CMS Global Pool: Scalability and Stability**
- **Scalability tests with OSG**
- **Next scalability bottlenecks and actions to mitigate them**
- **Future plans on scalability tests**
- **Stability and High Availability**



CMS Global Pool



- The CMS “Global Pool” is a **single HTCondor pool** covering all Grid computing processing **resources pledged to CMS**, plus significant Cloud and **opportunistic resources**.
- **Resource provisioning** by a **glideinWMS** frontend which talks to several glideinWMS factories to submit pilots to sites.
- **Jobs matched to pilots** with a HTCondor Negotiator.



Main limits to scalability:

- I/O between these components
- Combinatorics of Negotiator (jobs x pilot startd's)
- Speed of individual components

See backup slides for HW specs



Scalability and Stability



- The **challenge** we face is getting the **Global Pool** (glideinWMS & HTCondor) to work stably at higher and **higher scales** each year, e.g. **+40%** year over year between 2015 and 2016.
- Currently CMS has regular access to **~150,000 dedicated CPUs** (including spare cycles on the Tier-0 and HLT), plus another **~50,000 CPUs** for which we compete **opportunistically**.
- **Connecting new and diverse resource types** pushes the scale even faster: LPC at Fermilab, AWS, CMS HLT, HPC, etc.



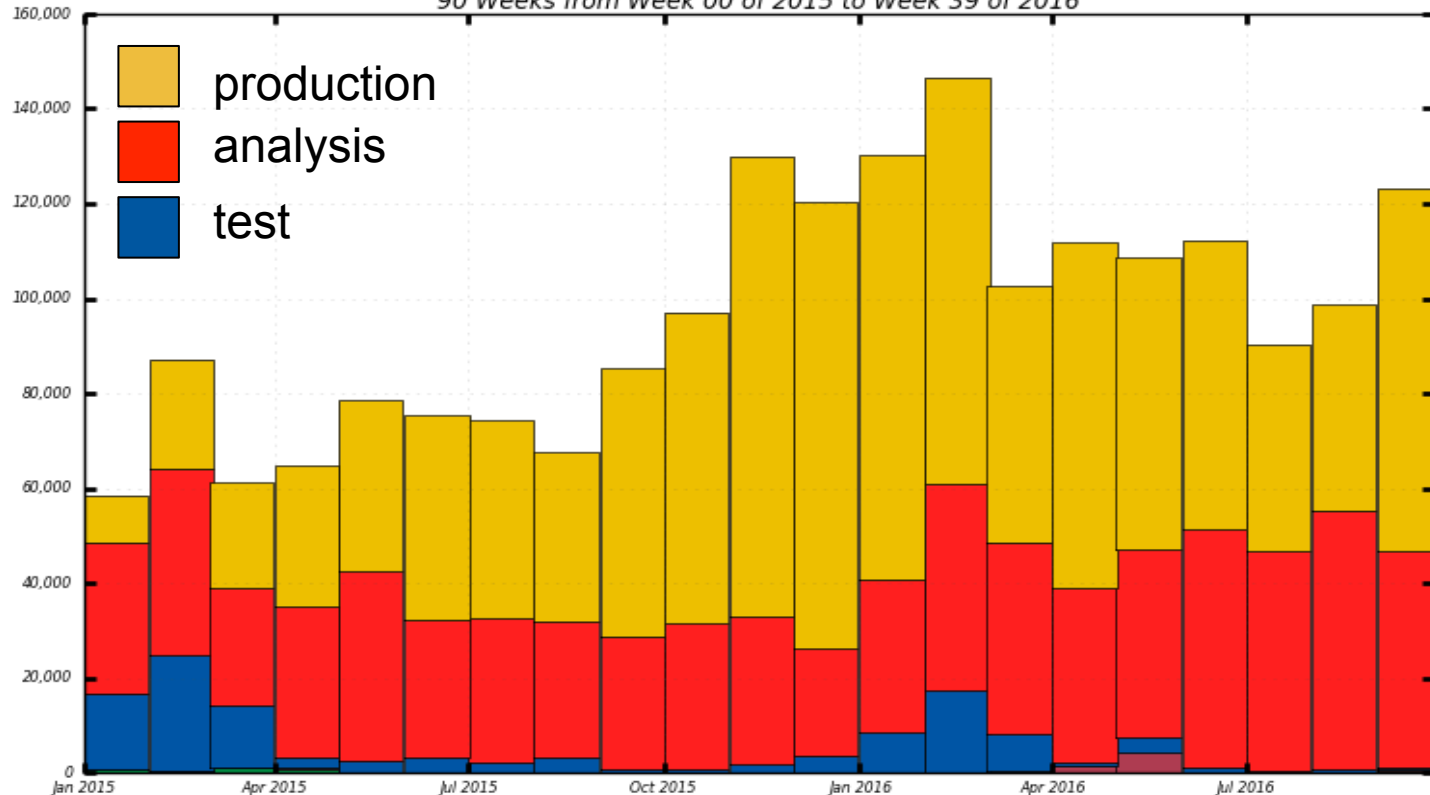
Higher scales during Run II



idashboard

Running Job Cores

90 Weeks from Week 00 of 2015 to Week 39 of 2016



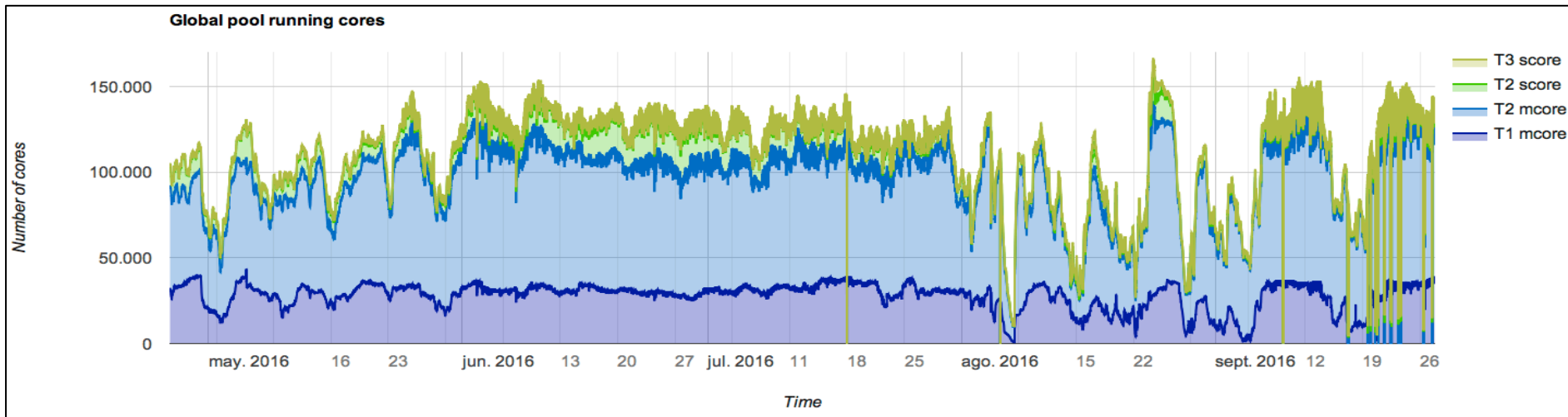
Monthly avg.
concurrently used
CPU cores
running CMS jobs
since the
beginning of Run
II (T0+T1s+T2s
+T3s).

Analysis has
doubled.

Production has
trebled.

Current scalability

- Instantaneous peaks of $\sim 160,000$ CPUs in the global pool (T1+T2+T3, including HLT)





Work with OSG

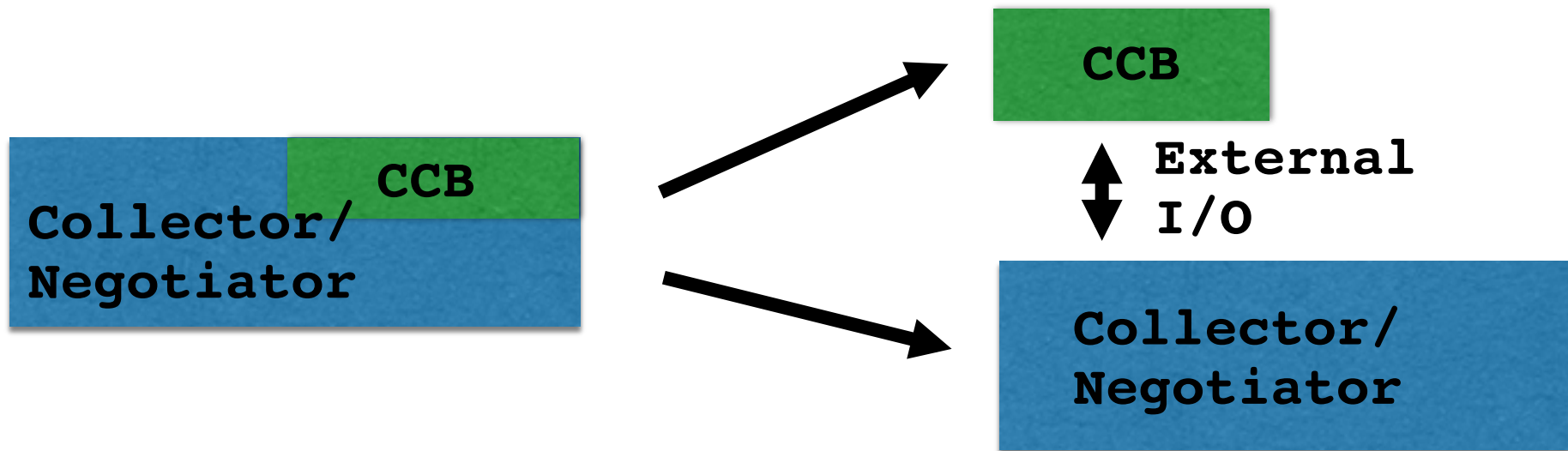


- Last CHEP we reported the results of scale testing to **200,000 CPUs** with the OSG [*]. Used single-CPU “über-glideins” with multiple of 32 startd’s per physical CPU.
- Main recommendation from that round was to put the HTCondor Communication Broker (**CCB**) on **separate hardware** from the rest of the Central Manager (I/O improvement).
- **Worked** closely with the **glideinWMS developers** during 2015 to make this happen.
- Resulted in a **much more stable Central Manager**, capable of easily reaching scales of 140,000 CPUs in the Global Pool.

[*] See CHEP-2015 contributions:

- “Pushing HTCondor and glideinWMS to 200K+ Jobs in a Global Pool for CMS before LHC Run 2”, J Letts *et al*
- “How much higher can HTCondor fly”, E. Fajardo *et al*

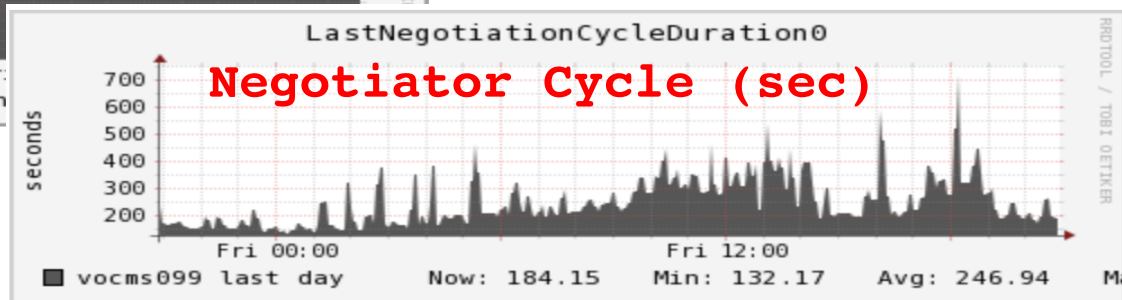
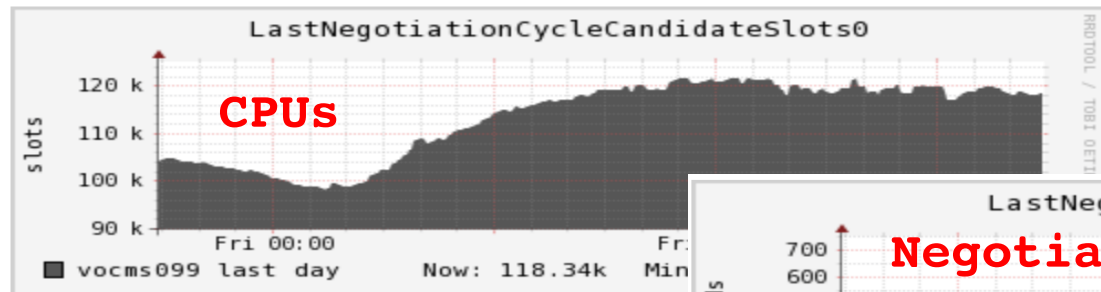
CMS Global pool Central Manager Hardware Split



- **Collector:** collecting all the information about the status of the HTCondor pool daemons (schedds and startds)
- **Negotiator:** responsible for all the match making within the HTCondor system

See backup slides for HW specs

Next bottleneck: the negotiator



- **Negotiation cycle**, the time it takes the Negotiator to match jobs to CPUs, is a main scalability limit.
- Both a **combinatoric** and **I/O** problem.
- **Negotiation cycle** was **blowing up** approaching **120K** CPUs.



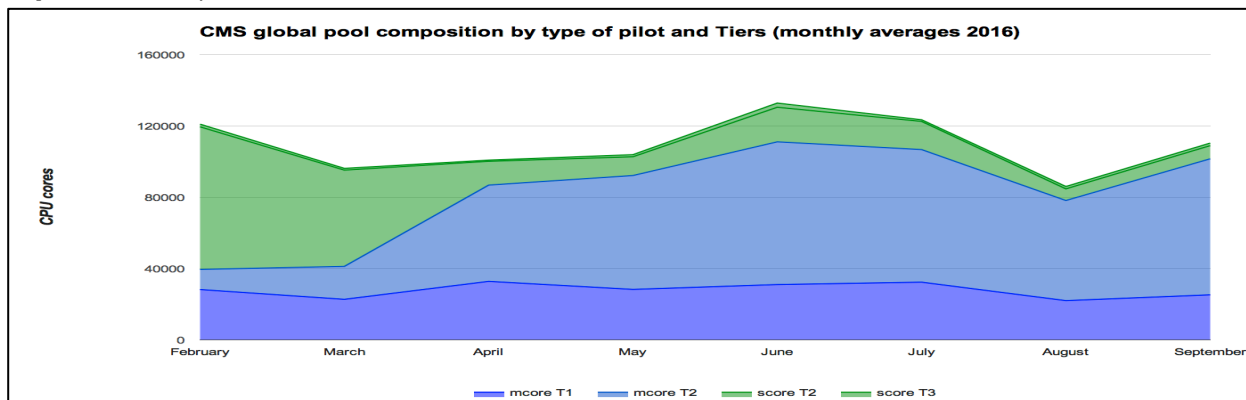
Next bottleneck: the negotiator

- We worked closely with the HTCondor developers to test and integrate a **prototype parallel Negotiator configuration**, where the **resources were split 3 ways** between various computing Tiers in the old Monarc Computing Model and geographic regions.
- Also allowed us to do **resource-based fair-share**, i.e. favor production at Tier-1 sites.
- HTCondor developers have since started working a **fully parallel Negotiator**, work which CMS is following closely.



Multi-core global pool

- Another of the most important scalability improvements has been the **move to multi-core pilots** (generally 8 CPUs) **and jobs**.
- Not only is **job memory requirement per CPU reduced**, it also reduces the **scale of the job/pilot matchmaking** by the number of CPUs per job.
- However, **more CPUs per pilot increased the slot status update rate** in the HTCondor Central Managers by the same factor of 8.
 - Traded **Negotiator improvement for I/O headache**.
- Worked closely with HTCondor developers to **filter out non-essential updates**, improving Negotiator cycle times by a factor of 2 or more (**I/O improvement**).





Other I/O improvements



- Moved the **job schedulers** (schedd's) to their own **dedicated child Collector**, reducing load on the top Collector.
- All child Collectors use **UDP** to faster update the top Collector **instead of TCP**.
- **Negotiator** fetches **multiple resource request lists** from the schedd's at a time.



Speedier components: schedds



- **Slow schedd's were a major blocker** for fast negotiation.
- **Reverted to 32-bit (not 64-bit) shadow binaries** for **lighter memory usage** on schedd's
 - (the shadow is the process that runs on the schedd to communicate with the remotely running job).
- **Disabled fsync()** on schedd to prevent blocking disk I/O.



Speedier components: negotiator



- Requested new configuration knobs for the Negotiator from the HTCondor developers to **drop overloaded and/or unresponsive schedd's**.
- Negotiator now **ignores glideins** (pilots) with **zero CPUs available**.



Future Plans

- We have planned a **new round of scale tests** with the **OSG** for the fourth quarter of 2016.
- Interested in examining
 - even **higher scales**
 - effects of **multi-core pilots**
 - a **more diverse job mix** (resource request lists)

to model **actual and future CMS usage**.

- **Look for scaling limitations and stability issues** well above 150,000 CPUs so that we can start working on them **now**, not when we encounter them next year!

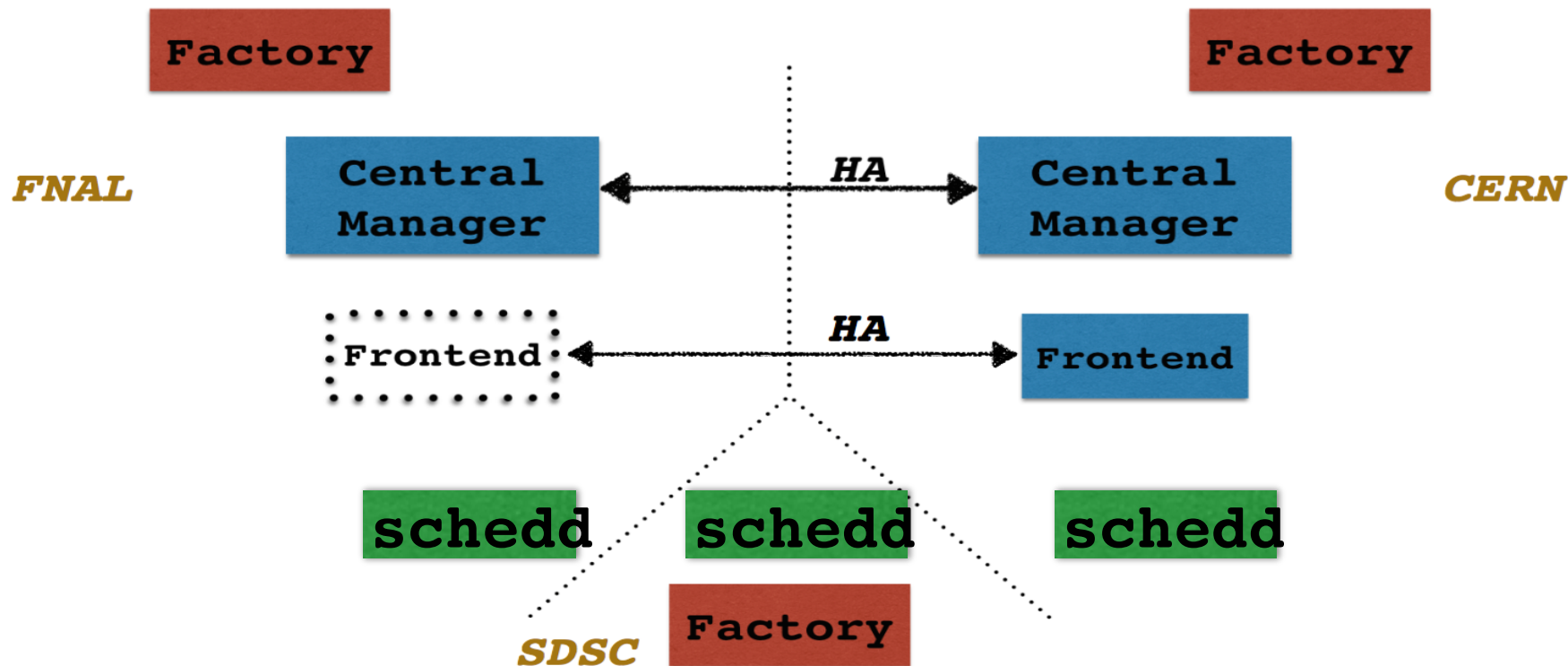


On Stability



- All HTCondor and glideinWMS services (Central Managers, CCBs, job schedulers, glideinWMS frontend and factories) are deployed in **high-availability** (HA) mode or in a **redundant set-up** in several “availability zones” now.
- Worked closely with the glideinWMS developers to test and integrate the new **HA-Frontend**.
- In case of (planned or unplanned) service interruptions at **CERN**, the central managers and frontend at **Fermilab** take over **running of the Pool**.

HA & Redundancy



See backup slides for HW specs



Conclusions & Outlook



- Key to the success of the Global Pool stably reaching ever higher scales has been **CMS's close coordination** with the **HTCondor** developers, the **glideinWMS** developers, and the **OSG**.
- We have regular meetings where we discuss the **priority for future work** and **report** back on our testing of **new features**.
- Thanks to the work of the OSG we can **anticipate future scaling and stability problems** and stay off the bleeding edge of limitations.

Extra slides



Global pool HW infrastructure



- GlideinWMS **Front-End** and **Factory**, HTCondor **Central Manager** and **CCB** at **CERN**
 - 24 core, 48 GB VMs, running on hypervisors with 10 Gbps ethernet connectivity
- Equivalent infrastructure for **HA at FNAL**
- Additional glideinWMS **factories** at **UCSD** and **OSG-GOC**
- **Submit nodes** (schedds):
 - Production schedds (~20), at CERN and FNAL
 - physical nodes 16 cores 64 GB, also 24 cores 48 GB
 - Analysis (CRAB) schedds (~10), at CERN and UCSD
 - 24 core 48 GB moving to 32 cores 58 GB VMs



Related CMS contributions at CHEP'16



- CMS readiness for multi-core workload scheduling
- CMS use of allocation based HPC resources
- Experience in using commercial clouds in CMS
- Software and Experience with Managing Workflows for the Computing Operation of the CMS Experiment
- CMS Event Processing Multi-core Efficiency Status
- Effective HTCondor-based monitoring system for CMS (poster)
- Connecting restricted, high-availability, or low-latency resources to a seamless Global Pool for CMS (poster)
- CMS Connect (poster)
- The CMS HLT as a dynamic cloud resource (poster)



References



- GlideinWMS: <http://www.uscms.org/SoftwareComputing/Grid/WMS/glideinWMS/doc.prd/index.html>
- HTCondor: <http://research.cs.wisc.edu/htcondor/>
- The pilot way to Grid resources using glideinWMS, I. Sfiligoi et al. <http://dx.doi.org/10.1109/CSIE.2009.950>
- Pushing HTCondor and glideinWMS to 200K+ Jobs in a Global Pool for CMS before LHC Run 2, J Letts et al. <http://dx.doi.org/10.1088/1742-6596/664/6/062030>
- How much higher can HTCondor fly, E. Fajardo et al, <http://dx.doi.org/10.1088/1742-6596/664/6/062014>



Abstract

The CMS Global Pool, based on HTCondor and glideinWMS, is the main computing resource provisioning system for all CMS workflows, including analysis, Monte Carlo production, and detector data reprocessing activities. Total resources at Tier-1 and Tier-2 sites pledged to CMS exceed 100,000 CPU cores, and another 50,000-100,000 CPU cores are available opportunistically, pushing the needs of the Global Pool to higher scales each year. These resources are becoming more diverse in their accessibility and configuration over time. Furthermore, the challenge of stably running at higher and higher scales while introducing new modes of operation such as multi-core pilots, as well as the chaotic nature of physics analysis workflows, place huge strains on the submission infrastructure. This paper details some of the most important challenges to scalability and stability that the Global Pool has faced since the beginning of the LHC Run II and how they were overcome.